

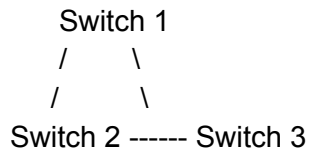
Sure. These two terms are **very important in switching**, especially when learning **STP (Spanning Tree Protocol)**. Let's understand them with a simple example.

1. Broadcast Storm

A **broadcast storm** happens when broadcast frames keep circulating through a network repeatedly, causing huge amounts of unnecessary traffic.

Simple example

Imagine you have:



Suppose **STP is not working** and there is a loop.

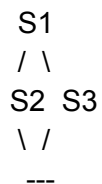
Now PC-A sends a broadcast:

PC-A → FF:FF:FF:FF:FF:FF

Switch 1 receives it.

Because it's a broadcast, Switch 1 forwards it out **all ports except the incoming port**.

So:



The frame can travel:

S1 → S2 → S3 → S1 → S2 → S3 → ...

There is nothing telling the switches:

"Stop! This frame has already been here."

So the same broadcast keeps circulating.

Result

Traffic becomes enormous:

Normal traffic



Broadcast



Loop



Repeated broadcasts



Network traffic increases massively



Switches/links become overloaded



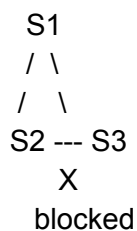
Network becomes slow or unusable

That's a **broadcast storm**.

Why does STP exist?

This is one of the main reasons we use **STP**.

STP detects Layer-2 loops and blocks a redundant path:



Now the broadcast has only one logical path, so it doesn't circulate forever.

2. MAC Address Flapping

MAC address flapping means a switch keeps learning the **same MAC address on different ports repeatedly**.

Remember the switch MAC address table:

MAC Address	Port
AA:AA:AA:AA:AA:AA	Fa0/1
BB:BB:BB:BB:BB:BB	Fa0/2

Suppose PC-A has:

MAC = AA:AA:AA:AA:AA:AA

Normally the switch learns:

AA:AA:AA:AA:AA:AA → Fa0/1

But because of a Layer-2 loop, the same frame may arrive from another port.

The switch now thinks:

AA:AA:AA:AA:AA:AA → Fa0/2

Then another copy arrives from Fa0/1:

AA:AA:AA:AA:AA:AA → Fa0/1

Then from Fa0/2:

AA:AA:AA:AA:AA:AA → Fa0/2

So the switch keeps changing its MAC table:

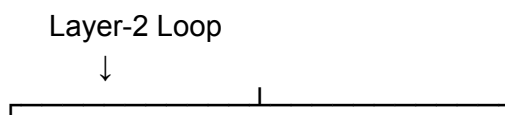
Fa0/1 → Fa0/2 → Fa0/1 → Fa0/2 → ...

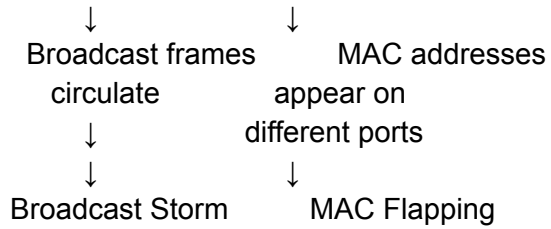
This is **MAC address flapping**.

The connection between them

These two problems are often caused by the **same underlying issue: a Layer-2 loop**.

Think of it like this:





Easy way to remember

Term	What happens?
Broadcast Storm	Broadcast frames keep circulating
MAC Flapping	Same MAC keeps moving between switch ports
Main cause	Layer-2 switching loop
Main protection	STP

Real-world scenario

Suppose you accidentally connect two switches with **two cables**:

Switch 1 ===== Switch 2
Cable 1

Switch 1 ===== Switch 2
Cable 2

Without STP, you've created a loop.

That can cause:

1. Broadcast Storm

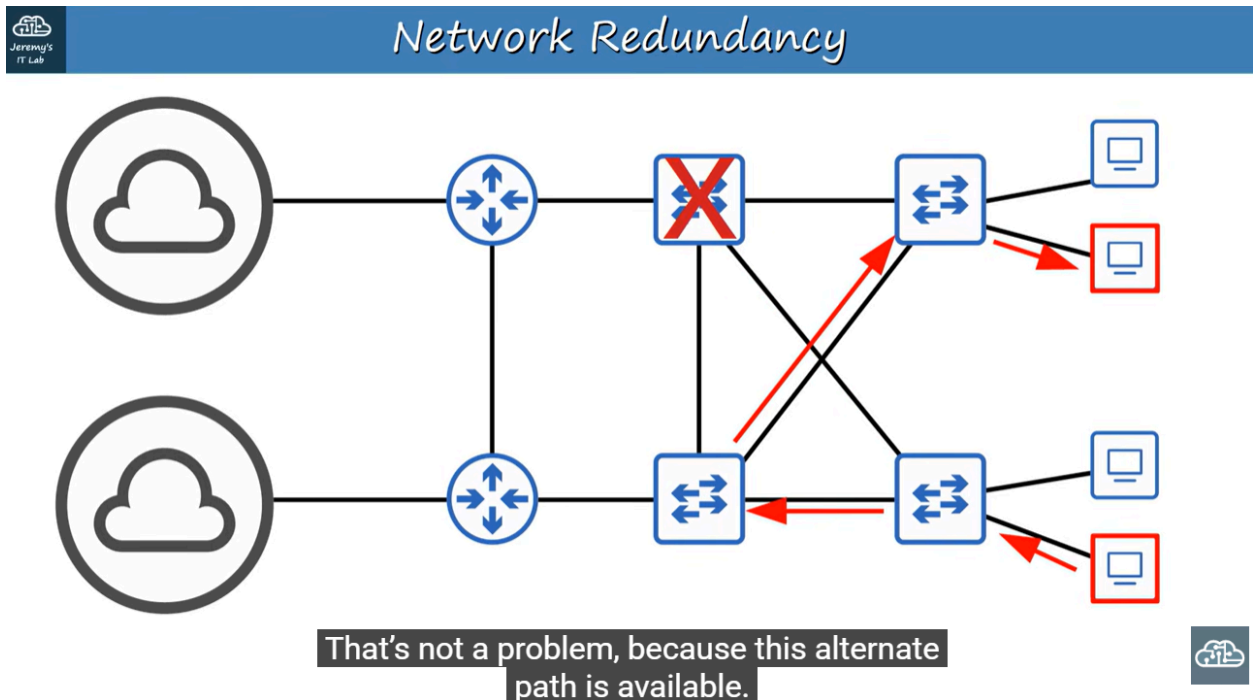
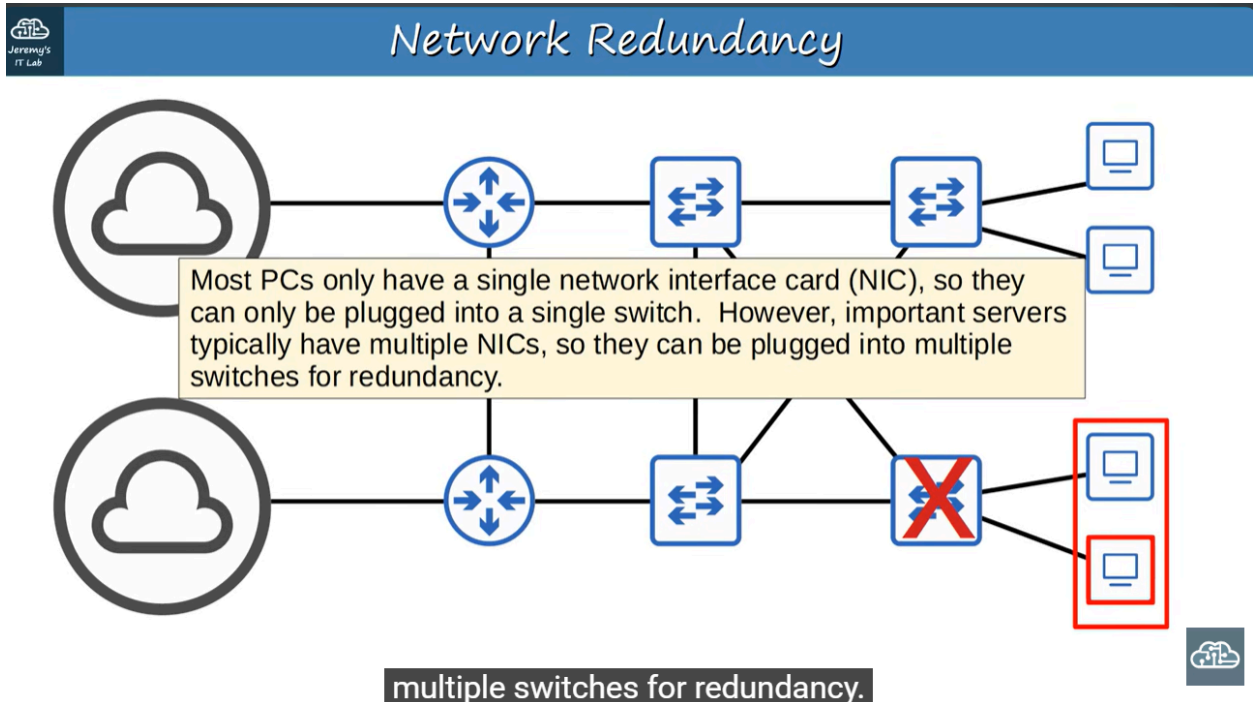
Broadcast → S1 → S2 → S1 → S2 → ...

2. MAC Flapping

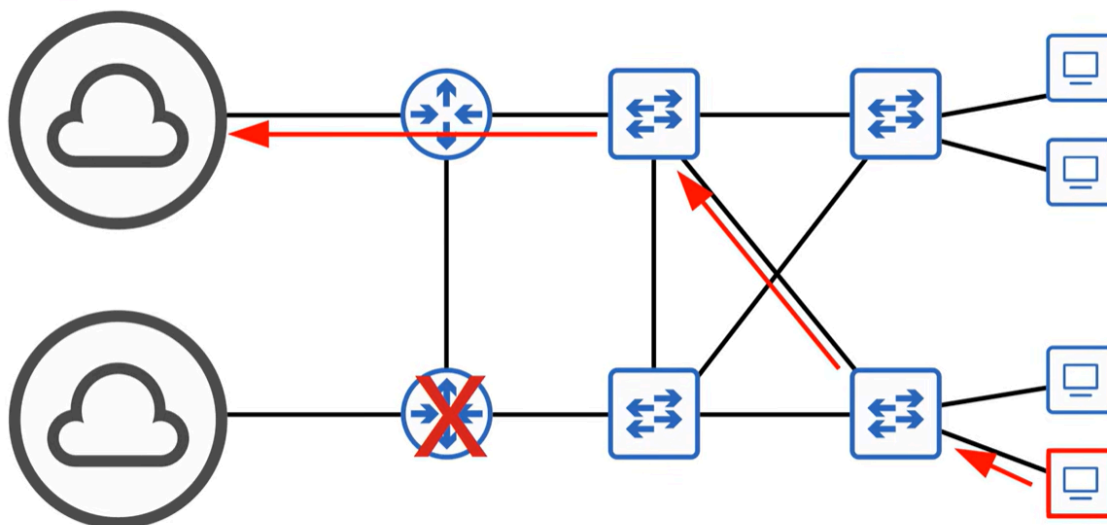
MAC AA → Port 1
MAC AA → Port 2
MAC AA → Port 1
MAC AA → Port 2

...

So if you see **broadcast storms + MAC flapping**, one of the first things to investigate is a **Layer-2 loop/STP problem**.



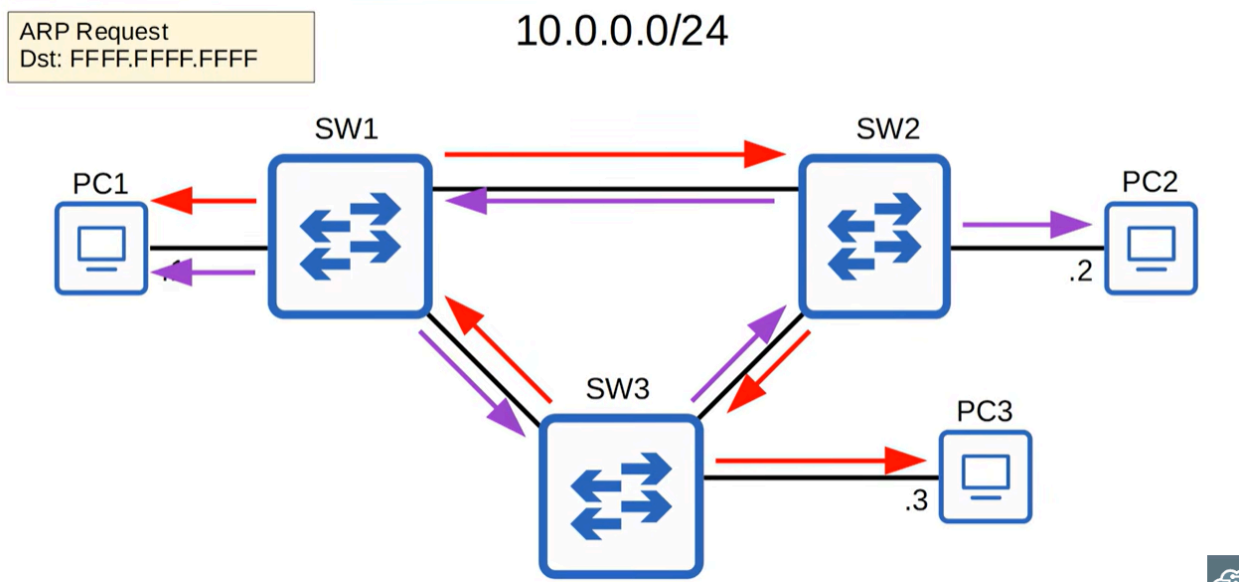
Network Redundancy



reach the Internet via this or another alternate path.



- Redundancy is an essential part of network design.
- Modern networks are expected to run 24/7/365. Even a short downtime can be disastrous for a business.
- If one network component fails, you must ensure that other components will take over with little or no downtime.
- As much as possible, you must implement redundancy at every possible point in the network.



Exactly. Let's use **your diagram** and connect it directly to the **two concepts we just learned: Broadcast Storm and MAC Address Flapping.**

We have:

PC1 = 10.0.0.1

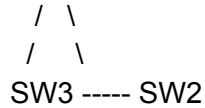
PC2 = 10.0.0.2

PC3 = 10.0.0.3

Network = 10.0.0.0/24

And the switches form a triangle:

SW1



That triangle is the important part because it creates a **Layer-2 redundant path/loop** if STP doesn't block one of the links.

Step 1 — PC1 wants to talk to PC2

PC1 knows:

PC1 IP = 10.0.0.1

PC2 IP = 10.0.0.2

Both are in:

10.0.0.0/24

So PC1 knows:

"PC2 is in my local network. I need PC2's MAC address."

Therefore PC1 sends an **ARP Request**.

Conceptually:

Who has 10.0.0.2?

Tell 10.0.0.1

The Ethernet destination is:

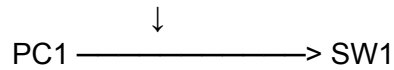
FF:FF:FF:FF:FF:FF

That's a **broadcast**.

Step 2 — PC1 → SW1

PC1 sends:

ARP Request



The destination MAC is:

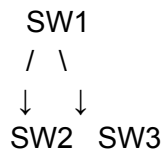
FFFF.FFFF.FFFF

SW1 says:

"This is a broadcast. I need to flood it."

So SW1 sends it out its other ports.

There are **two paths**:



So the ARP request travels:

PC1 → SW1 → SW2

and

PC1 → SW1 → SW3

This is normal behavior for a broadcast.

Step 3 — SW2 receives the broadcast

SW2 receives:

ARP Request
"Who has 10.0.0.2?"

PC2 is connected to SW2.

So SW2 forwards the broadcast toward PC2:

PC1 → SW1 → SW2 → PC2

PC2 receives it and says:

"That's my IP address!"

PC2 knows PC1's IP and MAC from the ARP request.

Step 4 — PC2 sends ARP Reply

PC2 now sends an **ARP Reply**.

Unlike the ARP Request, the reply can be **unicast** because PC2 now knows PC1's MAC address.

Conceptually:

PC2 → SW2 → SW1 → PC1

The ARP process is now complete.

PC1 learns:

10.0.0.2 → PC2's MAC address

For example:

10.0.0.2 → AA:AA:AA:AA:AA:02

Step 5 — PC1 can now send data to PC2

Suppose PC1 wants to send:

Hello PC2

PC1 creates an Ethernet frame:

Source MAC = PC1 MAC

Destination MAC = PC2 MAC

Now it is **unicast**, not broadcast.

Normally the path would be:

PC1

↓
SW1
↓
SW2
↓
PC2

That's the normal communication flow.

BUT... look at your topology 🧐

You have:

```
      SW1
     /  \
    /    \
   /      \
  SW3 --- SW2
```

There are **multiple paths** between SW1 and SW2.

For example:

Path 1

SW1 → SW2

Path 2

SW1 → SW3 → SW2

That creates a **Layer-2 loop**.

And this is where the two concepts we discussed become important.

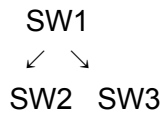
1. Broadcast Storm in YOUR diagram

Remember the ARP Request from PC1?

PC1 → SW1

It's a broadcast.

SW1 floods it:



Now SW3 also receives the broadcast.

SW3 thinks:

"This is a broadcast. I need to forward it out my other ports."

So SW3 sends it toward SW2.

PC1



SW1



SW3



SW2

But SW2 may then forward it toward SW1 because it's a broadcast.

SW2 → SW1

Now SW1 receives another copy.

And it can flood again.

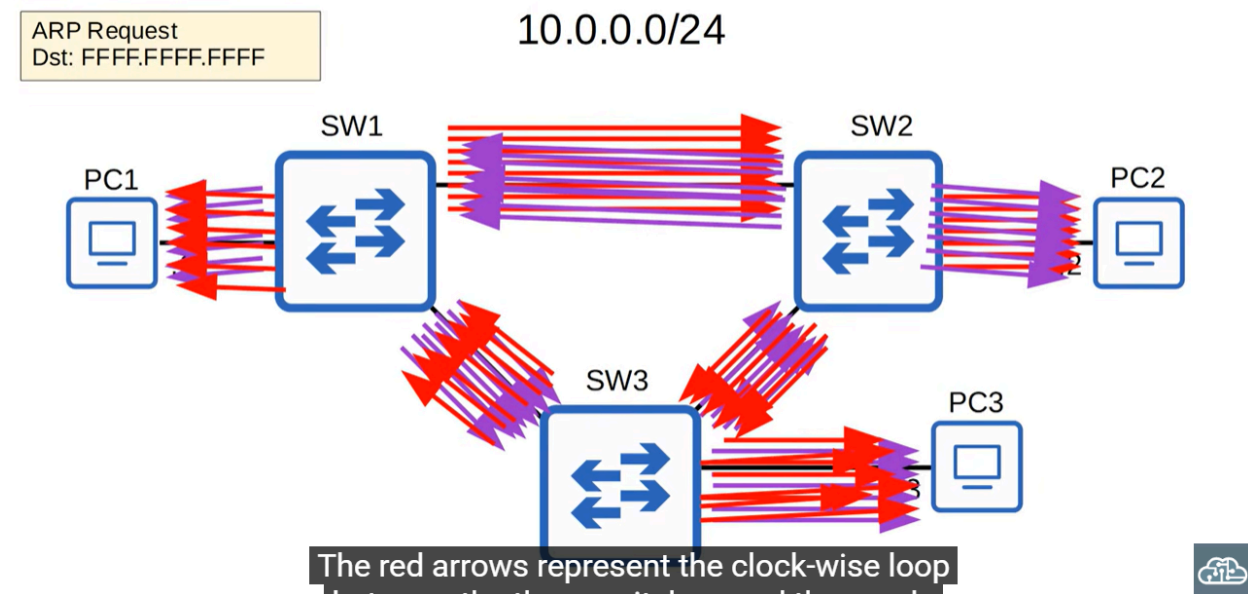
You can end up with:

SW1 → SW2 → SW3 → SW1 → SW2 → SW3 → ...

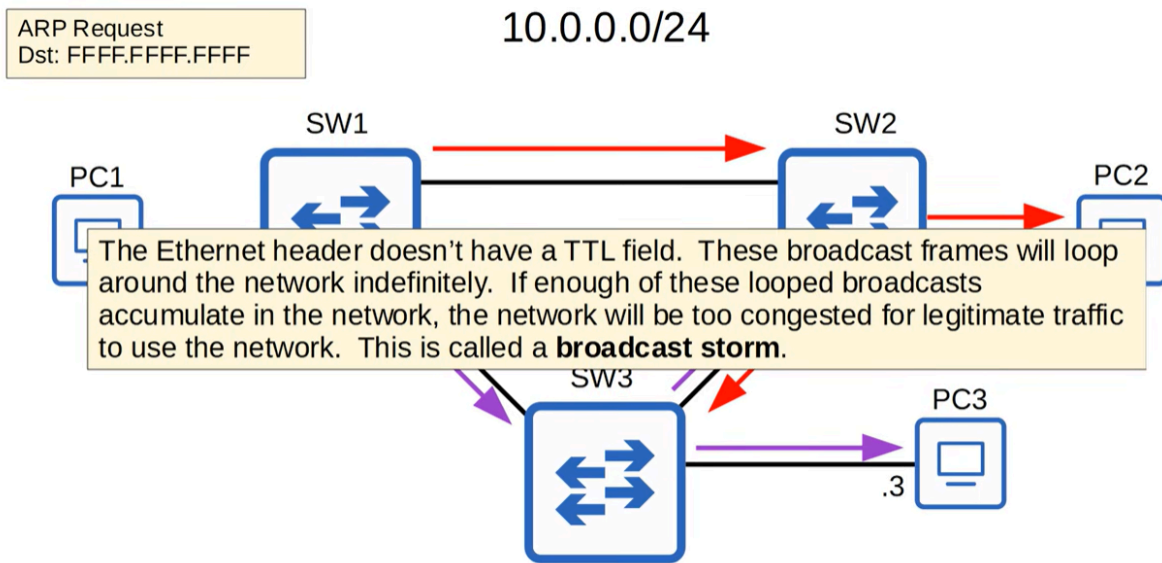
 **The broadcast keeps circulating.**

That's a **broadcast storm**.

The network gets flooded with copies of the same broadcast.



Broadcast Storms



2. MAC Address Flapping in YOUR diagram

Now let's look at the other problem.

Switches learn MAC addresses based on the **source MAC address** of incoming frames.

Suppose PC1's MAC is:

AAAA.AAAA.AAAA

SW1 initially learns:

AAAA.AAAA.AAAA → PC1 port

Good.

But because of the loop, duplicate frames can come back through different switch ports.

For example:

SW1 receives PC1's MAC
from its port toward SW2

So SW1 thinks:

AAAA.AAAA.AAAA → Port toward SW2

Then another copy of the frame comes back through SW3.

SW1 thinks:

AAAA.AAAA.AAAA → Port toward SW3

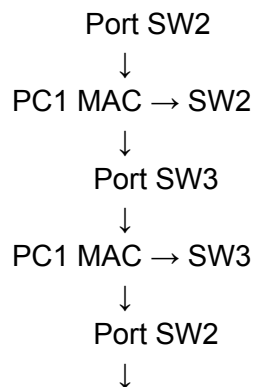
Then another copy comes from SW2:

AAAA.AAAA.AAAA → Port toward SW2

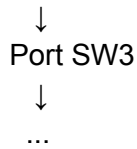
Then SW3:

AAAA.AAAA.AAAA → Port toward SW3

So the MAC table keeps changing:



PC1 MAC → SW2



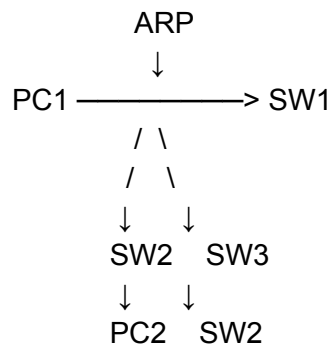
This is **MAC address flapping**.

The switch is basically saying:

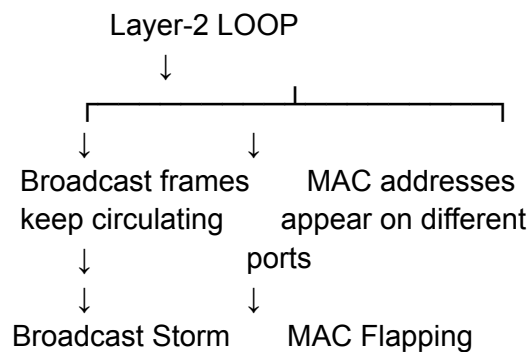
"I learned this MAC on SW2... wait, now I see it on SW3... wait, now SW2 again!"

So remember the complete story

When PC1 wants to communicate with PC2:

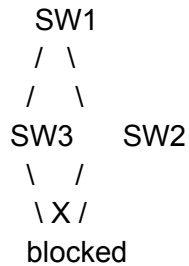


Because the switches form a triangle, without STP you can get:



And this is exactly why STP is important.

STP will detect the redundant path and logically block one link:



Then the topology becomes **loop-free**, while still keeping the redundant link available as a backup.

One-line memory trick:

Broadcast Storm = broadcast keeps circulating.

MAC Flapping = same MAC keeps appearing on different ports.

STP = prevents the Layer-2 loop causing both.

BPDU — Bridge Protocol Data Unit

Full form:

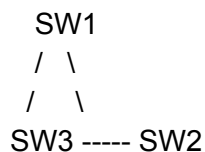
BPDU = Bridge Protocol Data Unit

A BPDU is a **special control frame used by switches running STP/RSTP to communicate with each other and prevent Layer-2 loops.**

Think of it as **STP's message between switches.**

1. Why do we need BPDU?

Look at your topology:



There are multiple paths between the switches.

Without STP:

SW1 → SW2 → SW3 → SW1 → SW2 → ...

A Layer-2 loop can cause:

- 🔥 Broadcast storm
- 🔄 MAC address flapping
- Network instability

So switches need to communicate and decide:

"Which switch should be the Root Bridge?"

"Which path is the best path to the Root?"

"Which port should forward?"

"Which redundant port should be blocked?"

BPDUs carry the information needed for these decisions.

2. What does a BPDU contain?

A simplified BPDU contains information like:

BPDU	
Root Bridge ID	
Root Path Cost	
Sender/Bridge ID	
Port ID	
Timers	
STP control information	

The important ones for you are:

Root Bridge ID

"Which switch do I believe is the Root?"

Root Path Cost

"What is my total path cost to the Root?"

Bridge ID

"Which switch am I?"

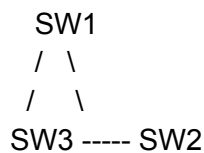
Port ID

"Which port is sending this BPDU?"

The switches compare this information to build the spanning tree.

3. How does BPDU actually work?

Let's take:



Initially, each switch can consider itself the Root.

So they exchange BPDUs.

For example:

SW1 ——— BPDU ———> SW2

SW1 ——— BPDU ———> SW3

SW2 ——— BPDU ———> SW1

SW2 ——— BPDU ———> SW3

SW3 ——— BPDU ———> SW1

SW3 ——— BPDU ———> SW2

Each switch examines the BPDU information.

4. Root Bridge election

Suppose:

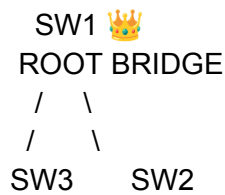
SW1 → Bridge ID = 32768 + AAAA
SW2 → Bridge ID = 32768 + BBBB
SW3 → Bridge ID = 32768 + CCCC

STP chooses the **lowest Bridge ID**.

Therefore:

AAAA < BBBB < CCCC

So:



The BPDUs allowed the switches to learn this information.

5. After Root Bridge election

Now SW2 asks:

"What is my best path to SW1?"

Suppose:

SW2 → SW1 = cost 19

and:

SW2 → SW3 → SW1
= 19 + 19
= 38

So SW2 chooses:

SW2 → SW1

as its best path.

That port becomes the **Root Port**.

Similarly, SW3 finds its best path to SW1.

6. What happens to the redundant link?

Now we have:

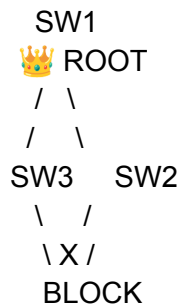


Both SW2 and SW3 already have good paths to SW1.

Therefore the SW3-SW2 link is redundant.

STP uses the BPDU information to determine which side should remain forwarding and which side should be blocked/discarding.

Conceptually:



Now:

SW1 → SW2

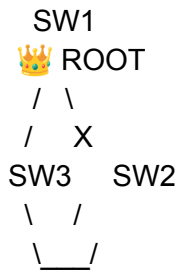
is the active path, while the redundant path is not forwarding.

The physical cable is still there.

7. What happens if the active link fails?

This is one of the biggest benefits of STP.

Suppose:



The SW1-SW2 link fails.

The switches exchange BPDUs again and recognize that the topology has changed.

STP can then allow the previously blocked path to become active:

SW1 → SW3 → SW2

So STP provides:

Redundancy + Loop prevention

8. BPDU and the STP states you asked about

Now connect this with your previous question.

Blocking

BPDU processing ✓

MAC learning ✗

Normal data forwarding ✗

The port can receive/process BPDUs but isn't forwarding normal user traffic.

Listening

The switch is actively processing BPDUs and determining the STP topology.

BPDUs processing ✓

MAC learning ✗

Data forwarding ✗

It is essentially asking:

"What is the Root? What should my port role be?"

Learning

The STP decision has been made, and the switch begins learning MAC addresses.

BPDUs processing ✓

MAC learning ✓

Data forwarding ✗

For example:

PC1 MAC → Fa0/1

PC2 MAC → Fa0/2

Forwarding

Everything is active:

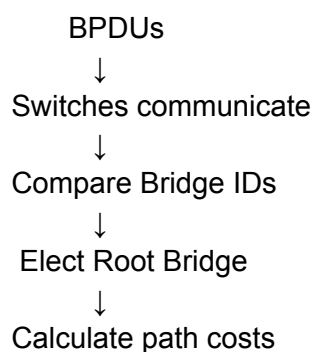
BPDUs processing ✓

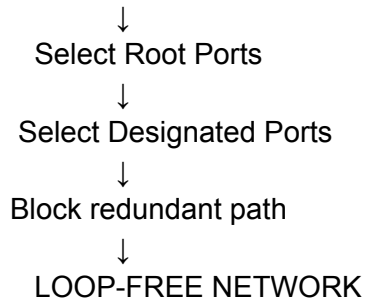
MAC learning ✓

Data forwarding ✓



The whole concept in one picture





Remember this:

BPDU = Bridge Protocol Data Unit

Purpose:

BPDUs allow switches to exchange STP information so they can build a loop-free Layer-2 topology.

And the easiest mental model is:

BPDU = "STP information packet that switches use to talk to each other."

One subtle point: **BPDUs are not ordinary user-data packets like your PC1 → PC2 traffic.**
They are **control frames generated for STP itself.**

STP



Spanning Tree Protocol

- 'Classic Spanning Tree Protocol' is IEEE 802.1D.
- Switches from ALL vendors run STP by default.
- STP prevents Layer 2 loops by placing redundant ports in a blocking state, essentially disabling the interface.
- These interfaces act as backups that can enter a forwarding state if an active (=currently forwarding) interface fails.
- Interfaces in a forwarding state behave normally. They send and receive all normal traffic.
- Interfaces in a blocking state only send or receive STP messages (called BPDUs = Bridge Protocol Data Units).



Spanning Tree Protocol

- By selecting which ports are **forwarding** and which ports are **blocking**, STP creates a single path to/from each point in the network. This prevents Layer 2 loops.
- There is a set process that STP uses to determine which ports should be forwarding and which should be blocking.
- STP-enabled switches send/receive Hello BPDUs out of all interfaces, the default timer is 2 seconds (the switch will send a Hello BPDU out of every interface, once every 2 seconds).
- If a switch receives a Hello BPDU on an interface, it knows that interface is connected to another switch (routers, PCs, etc. do not use STP, so they do not send Hello BPDUs).



Process For Stp



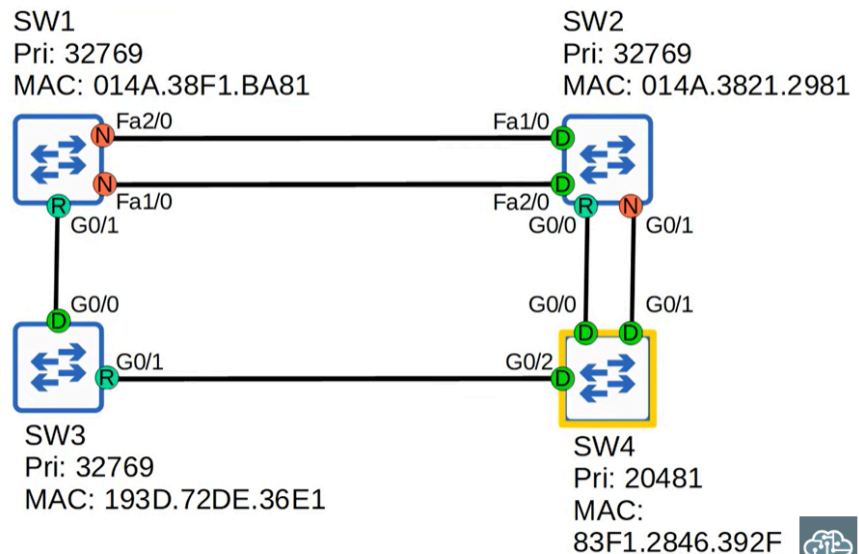
Spanning Tree Protocol

- 1) One switch is elected as the root bridge. All ports on the root bridge are **designated ports** (forwarding state). Root bridge selection:
1: Lowest bridge ID
- 2) Each remaining switch will select ONE of its interfaces to be its **root port** (forwarding state). Ports across from the root port are always **designated ports**. Root port selection:
1: Lowest root cost
2: Lowest neighbor bridge ID
3: Lowest neighbor port ID
- 3) Each remaining collision domain will select ONE interface to be a **designated port** (forwarding state). The other port in the collision domain will be **non-designated** (blocking). Designated port selection:
1: Interface on switch with lowest root cost
2: Interface on switch with lowest bridge ID



Spanning Tree Quiz 6

Identify the root bridge, and the role of each interface on each switch in the network (root/designated/non-designated)



Speed	STP Cost
10 Mbps	100
100 Mbps	19
1 Gbps	4
10 Gbps	2

STP — Complete Reference Notes

You already understand the basic idea of STP quite well. The areas where your questions showed a gap were:

- 1. BPDU flow through a blocked port
- 2. What Max Age actually means
- 3. Difference between STP port role and port state
- 4. What happens during failure/recalculation
- 5. Why Listening and Learning are separate
- 6. How STP provides redundancy without creating a loop

I'll make those especially clear below.

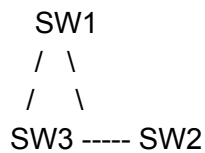
1. What is STP?

STP = Spanning Tree Protocol

STP is a Layer-2 protocol used by switches to:

1. Prevent Layer-2 loops
2. Prevent broadcast storms
3. Prevent MAC address instability/flapping
4. Maintain redundant paths
5. Provide a backup path if the active path fails

Without STP



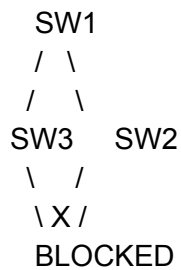
A broadcast could circulate:

SW1 → SW2 → SW3 → SW1 → SW2 → ...

🔥 Broadcast storm

🔄 MAC flapping

With STP



One redundant path is logically blocked.

2. The fundamental STP idea

This is the most important sentence in your notes:

STP does not remove redundant physical links. It logically blocks redundant forwarding paths so that only one logical path exists at a time.

If the active path fails:

Active path fails



STP detects topology change



Recalculates



Previously blocked path can become active

So STP gives:

Loop prevention + redundancy

3. BPDU

BPDU = Bridge Protocol Data Unit

BPDUs are special STP control frames exchanged between switches.

They contain information such as:

Root Bridge ID

Root Path Cost

Sender/Bridge ID

Port ID

STP timers

BPDUs allow switches to determine:

Who is Root?



What is my best path to Root?



Which port should forward?

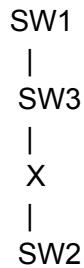


Which redundant port should block?

4. IMPORTANT: Blocked does NOT mean disconnected

This was one of your major doubts.

Suppose:



The SW3-SW2 port may be **blocking**.

But:

Normal data → 
MAC learning → 
BPDU processing → 

The physical link remains alive.

Why?

Because the switch needs to continue receiving STP information.

For example:

SW3 ——— BPDU ———> SW2

Even though normal data forwarding is blocked.

Therefore, if SW2 loses its active path to the Root, it can still receive STP information through the redundant connection.

Remember:

Blocked = no normal data forwarding, NOT "the cable is dead."

5. STP Port Roles vs Port States

This distinction is extremely important.

Port Role

Answers:

"What job does this port perform in the spanning tree?"

Main roles:

- Root Port
- Designated Port
- Alternate Port

Port State

Answers:

"What is this port currently allowed to do?"

Traditional STP states:

- Blocking
- Listening
- Learning
- Forwarding
- Disabled

Don't mix these two.

For example:

Port Role = Root Port

Port State = Forwarding

or:

Port Role = Alternate Port

Port State = Blocking

6. STP Port Roles

Root Bridge

The switch with the **lowest Bridge ID**.

SW1
👑 ROOT

The Root Bridge does not have a Root Port.

Root Port (RP)

On every **non-root switch**, the Root Port is the port providing the **best path toward the Root Bridge**.

Example:

SW2 ————— SW1
↑ ↑
RP ROOT

A non-root switch normally has **one Root Port**.

Designated Port (DP)

The forwarding port selected for a particular Layer-2 segment.

The Root Bridge's active ports are normally Designated Ports.

Alternate Port

A redundant path toward the Root.

It is not forwarding in the current topology.

In traditional STP terminology, you'll often hear this described as a **blocked port**.

7. STP Port States

Now we get to the states you specifically asked about.

Traditional **802.1D STP** has:

Blocking



Listening



Learning



Forwarding

Plus:

Disabled

8. Blocking State

The port is not forwarding normal data.

Normal data forwarding → 

MAC learning → 

BPDUs processing → 

Why receive BPDUs?

Because the redundant path must remain aware of STP changes.

Example:

SW3 — BPDUs —> SW2

even though:

SW3 — DATA —> SW2 

This is the concept you were missing earlier.

9. Listening State

The port is participating in STP topology calculation.

It processes BPDUs and determines things like:

Who is Root?

Which path is better?

What is my port role?

Should I forward or remain blocked?

But:

Normal data forwarding → 

MAC learning → 

BPDU processing → 

Why no MAC learning yet?

Because STP is still stabilizing the topology.

If the switch started forwarding and learning immediately, a temporary loop could occur.

So STP gives itself time to determine the tree.

10. Learning State

Now the port has been selected to participate in forwarding.

But it still doesn't forward normal data yet.

Instead, it begins learning source MAC addresses.

Normal data forwarding → 

MAC learning → 

BPDU processing → 

For example:

PC1 → Switch

The switch learns:

PC1 MAC → Fa0/1

● Very important

Learning does NOT mean STP is learning about loops.

It means:

The switch is learning MAC addresses for its MAC address table.

STP's loop-prevention decision comes from:

BPDUs + Bridge ID + Path Cost + port roles.

11. Forwarding State

Now everything is active.

Normal data forwarding → 

MAC learning → 

BPDU processing → 

PC1 can communicate with PC2 normally.

12. Disabled State

The interface is administratively disabled or otherwise not participating.

Data forwarding → 

MAC learning → 

BPDU processing → 

Example:

shutdown

13. Traditional STP Port-State Table

State	Data Forwarding	MAC Learning	BPDU
Blocking	✗	✗	✓
Listening	✗	✗	✓
Learning	✗	✓	✓
Forwarding	✓	✓	✓
Disabled	✗	✗	✗

14. Why Listening AND Learning?

This was another important area you were questioning.

Think:

Listening

"Let me understand the topology first."

BPDU → understand STP

Learning

"The topology is okay; let me learn MAC addresses before forwarding."

Source MAC → MAC table

Forwarding

"Now I can forward user traffic."

PC1 → PC2

So:

LISTENING

"I am deciding."



LEARNING

"I am preparing my MAC table."



FORWARDING

"I am forwarding traffic."

15. STP Timers

Traditional STP uses three important timers:

1. **Hello Time**
2. **Max Age**
3. **Forward Delay**

Default values commonly used:

Timer	Default
Hello	2 seconds
Max Age	20 seconds
Forward Delay	15 seconds

These are traditional 802.1D STP defaults.

16. Hello Timer

Hello Time = how frequently the Root Bridge sends BPDUs.

Default:

2 seconds

Conceptually:

Time 0 → BPDU

Time 2 → BPDU

Time 4 → BPDU

Time 6 → BPDU

...

Think:

Hello = "I'm still here."

The Root Bridge generates configuration BPDUs periodically.

17. Max Age

This is where you previously had an important misconception.

Max Age does NOT mean:

"After 20 seconds, automatically activate the backup path."

✗ Incorrect.

Instead:

Max Age determines how long a switch keeps old BPDU information before considering it expired if fresh information is not received.

Default:

20 seconds

Example:

SW1 ————— SW2

SW2 receives BPDUs from SW1.

Then:

SW1 ——— X ——— SW2

SW2 stops receiving them.

It doesn't immediately forget SW1.

It waits for the old information to age out.

No fresh BPDU



Max Age expires



Old information becomes invalid



STP recalculates

● Important correction

Max Age **doesn't itself activate the backup path.**

It causes old STP information to expire, after which STP can make new decisions.

18. Forward Delay

Traditional STP uses:

Forward Delay = 15 seconds

It is associated with the transition through:

Listening → Learning

Learning → Forwarding

The purpose is to give the topology time to stabilize before normal traffic forwarding begins.

In traditional STP, this contributes to the well-known convergence delay.

19. STP Timer Summary

Remember:

Hello

How often are BPDUs generated?

2 sec

Max Age

How long can old BPDUs remain valid?

20 sec

Forward Delay

How long does STP spend in the transitional states?

15 sec

Memory:

Hello = send

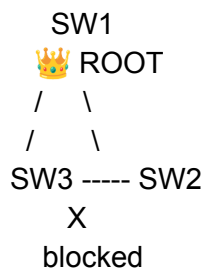
Max Age = expire

Forward Delay = transition

20. 🚫 What happens when a path fails?

This is the exact concept you were exploring earlier.

Suppose:



Active path:

SW1 → SW2

Backup:

SW1 → SW3 → SW2

Now:

SW1 —X— SW2

SW2 stops receiving BPDUs from its current Root Port.

Eventually the old information expires.

Then STP considers available information.

SW3 may still have valid information:

SW3 knows:

Root = SW1

The redundant path can then be selected as the new path toward the Root.

Important:

STP doesn't necessarily say:

"Everybody forget everything and elect a Root from zero."

Instead, the switches exchange new STP information and **recalculate the affected topology**.

If the original Root is still reachable, it can remain Root.

21. STP Convergence

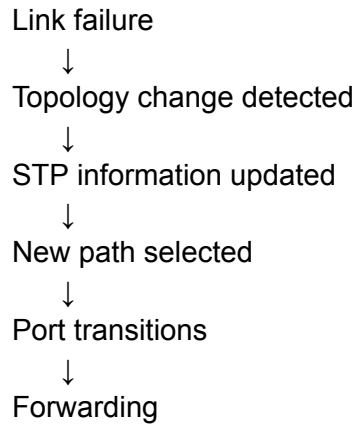
Convergence means:

The network reaches a new stable STP topology after a change.

Example:

Normal topology





Traditional STP can take significant time to converge.

This is one reason **RSTP** was introduced.

22. RSTP

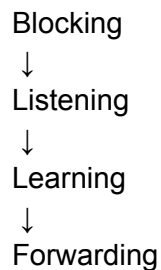
RSTP = Rapid Spanning Tree Protocol

Standard:

802.1w

RSTP improves convergence compared with traditional 802.1D STP.

Traditional STP:



RSTP simplifies the operational states to:

Discarding
Learning
Forwarding

RSTP also introduces faster mechanisms for detecting and agreeing on topology changes.

23. STP Protection Features

Now let's move to the features you asked about.

These are extremely useful in real Cisco networks:

PortFast
BPDU Guard
Root Guard
Loop Guard

They solve **different problems**.

24. PortFast

PortFast is normally used on a port connected to an **end device**, such as:

PC
Printer
Server

Example:

PC ——— Switch

Normally STP might take time going through its transitional process.

With PortFast, an access port can move to forwarding much faster.

Why?

Because a PC is not supposed to participate in the Layer-2 switching topology.

So we tell the switch:

"This port connects to an end device. Don't make the PC wait through normal STP convergence."

Important:

PortFast should generally be used toward end devices, not between switches.

25. BPDU Guard

BPDU Guard is closely related to PortFast.

Suppose:

PC ——— Switch

You enable PortFast.

You expect:

PC

But someone accidentally connects another switch:

SW2 ——— PortFast Port ——— SW1

Now SW1 receives a BPDU on a port that was expected to be an end-device port.

BPDU Guard can protect the network by putting the port into an error-disabled state.

Concept:

PortFast + BPDU Guard

Expected:

PC 

Unexpected:

Switch/BPDU 

Memory:

BPDU Guard = "If I receive a BPDU here, shut/protect the port."

26. Root Guard

Root Guard protects your chosen **Root Bridge**.

Suppose:



You don't want someone connecting a switch with a superior Bridge ID and causing it to become Root.

Root Guard says essentially:

"Don't allow this port to become part of a path toward a superior Root."

If superior BPDUs are received, the port can enter a **root-inconsistent** state.

This prevents an unexpected switch from taking over Root Bridge responsibilities through that port.

Memory:

Root Guard = protects the Root Bridge position.

27. Loop Guard

Loop Guard protects against certain situations where a switch **stops receiving BPDUs unexpectedly** on a port that should be receiving them.

This is especially useful on redundant links.

Without protection, a port could potentially transition inappropriately toward forwarding and contribute to a loop under certain failure conditions.

Loop Guard says:

"If expected BPDUs disappear, don't blindly start forwarding."

The port can enter a **loop-inconsistent** state instead.

Memory:

Loop Guard = protects against loops caused by unexpected loss of BPDUs.

28. PortFast vs BPDU Guard vs Root Guard vs Loop Guard

This table is worth memorizing:

Feature	Main Purpose
PortFast	Quickly move end-device port to forwarding
BPDU Guard	Protect PortFast edge port from unexpected BPDUs
Root Guard	Prevent an unexpected switch from becoming Root
Loop Guard	Prevent loops when expected BPDUs disappear

Easy memory:

PortFast

→ Fast edge port

BPDU Guard

→ Don't accept BPDUs here

Root Guard

→ Don't let you become Root

Loop Guard

→ Don't forward blindly when BPDUs disappear

29. STP Configuration Mode — Cisco

On Cisco switches, you can configure STP globally.

First:

Switch(config)# spanning-tree ?

Depending on IOS/platform, you'll work with commands such as:

spanning-tree mode ...

Common modes include:

PVST+

Rapid-PVST+

MST

For example:

Switch(config)# spanning-tree mode rapid-pvst

This enables **Rapid-PVST+**.

30. Selecting the Root Bridge

Suppose you have:

SW1

SW2

SW3

You want SW1 to become Root.

Cisco provides:

Switch(config)# spanning-tree vlan 10 root primary

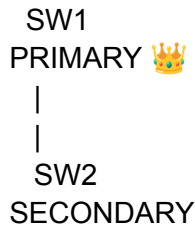
This tells the switch to adjust its STP parameters so it becomes the preferred Root for VLAN 10.

31. Root Secondary

You can configure another switch as the backup/preferred secondary:

Switch(config)# spanning-tree vlan 10 root secondary

Example:



If SW1 becomes unavailable, SW2 is positioned to become the Root.

Important:

"Primary" and "secondary" here are **STP root preference configurations**, not two simultaneously active Root Bridges for the same VLAN.

For a particular spanning-tree instance/VLAN, there is **one active Root Bridge at a time**.

32. 🔥 Two Root Bridges for Different VLANs

This is a very important concept for **STP load balancing**.

Suppose you have:



You have:

VLAN 10
VLAN 20

Instead of making SW1 Root for both VLANs, you can distribute the Root role:

VLAN 10 → SW1 Root
VLAN 20 → SW2 Root

This is possible because STP can maintain separate spanning-tree instances depending on the STP mode.

33. Why would we do this?

Suppose SW1 is Root for everything:

VLAN 10 → SW1

VLAN 20 → SW1

VLAN 30 → SW1

VLAN 40 → SW1

Traffic may heavily favor the same paths.

Instead:

VLAN 10 → SW1 Root

VLAN 20 → SW2 Root

Different VLANs can use different forwarding paths.

This is called:

STP Load Balancing

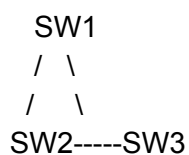
34. STP Load Balancing

Important:

STP load balancing does not mean both redundant links forward the same VLAN traffic simultaneously.

Instead, different VLANs can have **different STP trees**.

Example:



For VLAN 10:

SW1 = Root

For VLAN 20:

SW2 = Root

Therefore:

VLAN 10

→ prefers paths based on SW1's tree

VLAN 20

→ prefers paths based on SW2's tree

Traffic can be distributed across different physical links.

35. Cisco Example: VLAN Load Balancing

Suppose:

SW1

SW2

SW3

You could configure:

SW1

spanning-tree vlan 10 root primary

SW2

spanning-tree vlan 20 root primary

Then:

VLAN 10 → SW1 Root

VLAN 20 → SW2 Root

You can similarly configure secondary root preferences.

36. PVST+ and Rapid-PVST+

PVST+

Per-VLAN Spanning Tree Plus

Each VLAN has its own STP instance.

So:

VLAN 10 → Tree 10

VLAN 20 → Tree 20

VLAN 30 → Tree 30

This makes VLAN-based load balancing possible.

Rapid-PVST+

Rapid version of PVST+.

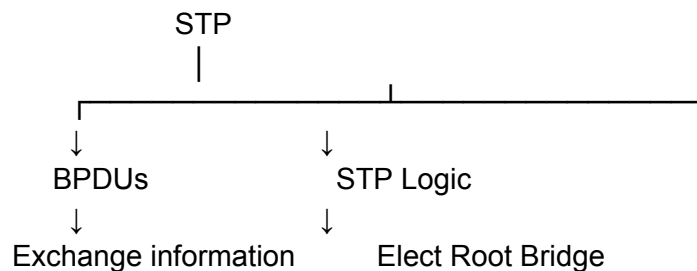
It provides:

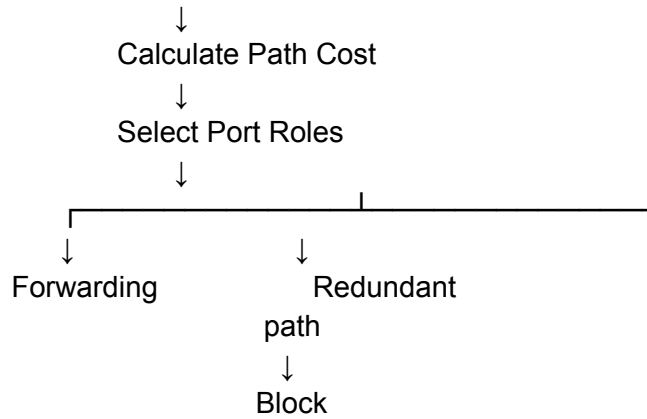
- Per-VLAN spanning tree
- Faster convergence
- RSTP behavior

Cisco networks commonly use **Rapid-PVST+** when using the Cisco per-VLAN approach.

37. The complete STP picture

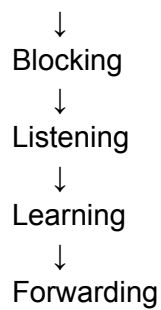
Put everything together:





Then:

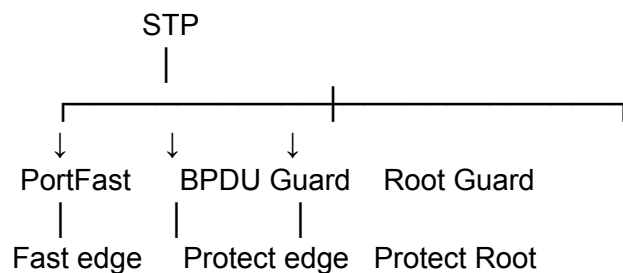
Port States



And timers control the traditional STP process:

Hello → BPDUs generation interval
 Max Age → old BPDU information expiry
 Forward Delay → state-transition timing

38. STP protection layer



And:

Loop Guard

↓
Protect against unexpected
loss of BPDUs causing loops

39. ★ The concepts you were previously missing

Based on the questions you've been asking, **these are the concepts I strongly recommend you remember:**

● 1. Blocking ≠ link down

Physical link = UP
Normal data = BLOCKED
BPDUs = still processed

This explains how a redundant link can later participate in STP.

● 2. BPDU ≠ normal data

PC1 → PC2

↓
Normal data

SW1 → SW2

↓
BPDU = STP control information

Don't mix these.

● 3. MAC learning ≠ loop detection

Learning state means:

Source MAC → switch MAC table

STP topology decisions come from:

BPDUs

+
Bridge ID
+
Path Cost
+
STP rules

● 4. Max Age ≠ backup activation timer

Incorrect:

Max Age expires
→ immediately activate backup

Correct:

No valid BPDU
→ old information ages out
→ STP recalculates
→ best available path may become active

● 5. STP doesn't necessarily restart from zero

A topology change doesn't mean:

Everyone forgets everything
→ everyone becomes Root
→ start from zero

Instead, STP is **distributed**.

Switches exchange updated information and the affected topology is recalculated.

● 6. Port Role ≠ Port State

ROLE:

Root
Designated
Alternate

STATE:

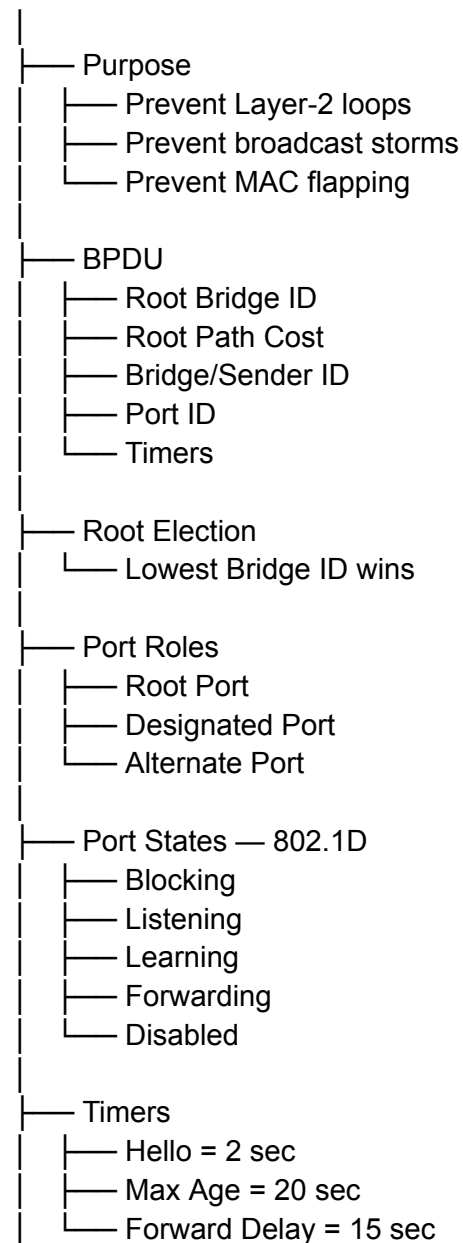
Blocking

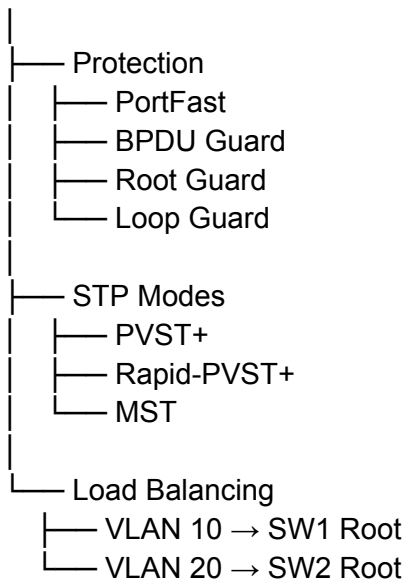
Listening
Learning
Forwarding

This distinction will save you from a lot of confusion in future STP questions.

40. Final STP Cheat Sheet

STP





The one mental model I want you to retain:

BPDUs build the STP picture → STP chooses the Root → switches choose the best paths → redundant paths are blocked → timers control traditional STP transitions → if a path fails, switches exchange updated information and a backup path can be brought into forwarding.

That is the **core STP story**. Once this is solid, Cisco STP commands become much easier to understand rather than just memorizing them.

STP — BPDU Guard & BPDU Filter Notes

1. What is a BPDU?

BPDU = Bridge Protocol Data Unit

BPDUs are STP messages exchanged between switches to:

- Elect the **Root Bridge**
- Calculate the **best path**
- Detect/prevent **Layer-2 loops**

Normally:

Switch ←—— BPDU ——→ Switch

2. BPDU Guard

Purpose

BPDU Guard protects a **PortFast/edge port** from an unexpected switch.

It basically says:

"This port should connect to an end device. If I receive a BPDU, shut the port down."

Example:

Switch

|

PortFast + BPDU Guard

|

PC

If somebody replaces the PC with another switch:

Switch

|

PortFast + BPDU Guard

|

Another Switch

↓

BPDU

↓

 BPDU Guard

↓

Err-disabled

Key point

BPDU Guard = Detect BPDU → Protect by shutting down the port.

3. BPDU Filter

Purpose

BPDU Filter is used to **suppress/filter BPDUs**.

Basically:

"Don't allow normal BPDU exchange on this port."

But there are **two ways to configure it**, and this was your main doubt.

4. Global BPDU Filter

Command

```
Switch(config)# spanning-tree portfast bpdupfilter default
```

This is a **global configuration**.

It applies to **PortFast ports**.

What happens?

Normally, BPDUs are filtered on the PortFast port.

But if the port actually **receives a BPDU**, the global BPDU Filter behavior changes:

BPDU received

↓

Global BPDU Filter stops filtering

↓

BPDU becomes visible to STP/BPDU Guard



BPDU Guard detects it



Interface → err-disabled

So global BPDU Filter can work together with BPDU Guard as a protection mechanism.

5. Interface-Level BPDU Filter

Command

```
Switch(config)# interface fa0/1
```

```
Switch(config-if)# spanning-tree bpduguard enable
```

This applies **only to that interface**.

If a BPDU arrives:

BPDU



Interface-level BPDU Filter



Ignored/filtered



BPDU Guard doesn't see it



 BPDU Guard is NOT triggered

This was your major question:

"If Filter blocks the BPDU, why do we need Guard?"

Answer:

Because **Guard's job is to detect the unexpected BPDU and shut the port.**

If interface-level filtering hides the BPDU from Guard, Guard cannot perform that protection.

6. Can BPDU Guard + BPDU Filter be enabled together?

YES 

They can be configured on the same port.

But **the behavior depends on how BPDU Filter is configured.**

Global Filter + Guard

Global BPDU Filter

+

BPDU Guard

↓

BPDU arrives

↓

Filter stops filtering

↓

Guard detects BPDU

↓

 Err-disabled

Interface Filter + Guard

Interface BPDU Filter

+

BPDU Guard

↓

BPDU arrives

↓

Filter ignores it

↓

Guard doesn't detect it

↓

 No err-disable

7. Global vs Interface Configuration

Global Configuration Mode

You are configuring a **general/default behavior**.

Switch(config)#

BPDU Guard globally

spanning-tree portfast bpduguard default

→ BPDU Guard is enabled by default on **PortFast ports**.

BPDU Filter globally

spanning-tree portfast bpdupfilter default

→ BPDU Filter is enabled by default on **PortFast ports**.

Interface Configuration Mode

You are configuring **one particular interface**.

```
Switch(config)# interface fa0/1
```

```
Switch(config-if)#
```

BPDU Guard

```
spanning-tree bpduguard enable
```

➡ Guard applies to **that interface**.

BPDU Filter

```
spanning-tree bpdufilter enable
```

➡ Filter applies to **that interface**.

8. Quick Comparison

Feature	BPDU Guard	BPDU Filter
Main purpose	Protection 	Suppress BPDUs 
Unexpected BPDU	Shuts port	Filters/ignores it depending on configuration
Global command	<code>spanning-tree portfast bpduguard default</code>	<code>spanning-tree portfast bpdufilter default</code>

Interface command	<code>spanning-tree bpduguard enable</code>	<code>spanning-tree bpdufilter enable</code>
-------------------	---	--

Global + BPDU received	Guard can trigger	Filter stops filtering when BPDU is detected
------------------------	-------------------	--

Interface-level + BPDU received	Guard may not see it if filtered	BPDU ignored
---------------------------------	----------------------------------	--------------

9. Your Main Doubt — Final Answer

You asked:

"If BPDU Filter blocks the BPDU, why do we need BPDU Guard?"

Think of them as two different approaches:

BPDU Guard

"Tell me if a switch appears here. If it does, shut the port."

BPDU Filter

"Don't let BPDUs operate through this port."

Therefore:

BPDU Guard → DETECT + SHUT DOWN 

BPDU Filter → SUPPRESS BPDU 

And the **important Cisco behavior** you learned is:

GLOBAL BPDU FILTER



BPDUs arrive



Filtering stops



BPDUs can trigger



ERR-DISABLED

versus:

INTERFACE BPDUs FILTER



BPDUs arrive



BPDUs ignored



BPDUs Guard doesn't trigger



30-Second Revision

Remember these 4 lines:

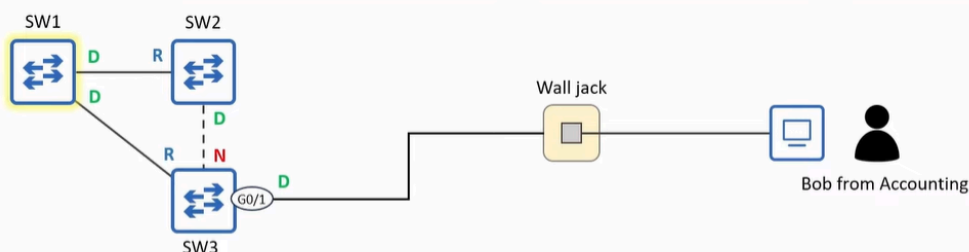
PortFast = Fast forwarding for an edge port.

BPDUs Guard = If BPDUs come → shut the port.

BPDUs Filter = Suppress BPDUs.

Global Filter + Guard ≠ Interface Filter + Guard — because global filtering can stop when a BPDUs is received, allowing Guard to detect it; interface-level filtering ignores the BPDUs, so Guard doesn't trigger.

That's the core of everything you asked in this discussion.



- My recommendation:
 - Enable PortFast and BPDU Guard however you prefer (per-port or by default).
 - Only enable BPDU Filter by default (global config mode).
 - unless you have a very good reason to enable it per-port

BPDU Guard and BPDU Filter can be enabled on the same port at the same time:

- If BPDU Filter is enabled in global config mode and the port receives a BPDU:
 1. BPDU Filter will be disabled.
 2. BPDU Guard will be triggered (and err-disable the interface).
- If BPDU Filter is enabled in interface config mode and the port receives a BPDU:
 - The BPDU will be ignored.
 - BPDU Guard will **not** be triggered.

- **PortFast** should only be enabled on ports connected to non-switch devices (end hosts, routers) that don't send BPDUs.
 - A PortFast-enabled port still sends BPDUs and will operate like a regular STP port if it receives BPDUs from a neighbor.
 - If an end user carelessly connects a switch to a port meant for end hosts, it could affect the STP topology
- **BPDU Guard** protects the network from unauthorized switches being connected to ports intended for end hosts.
 - If the port receives a BPDU, it enters the **error-disabled** (err-disabled) state, effectively disabling the port.
 - Per-port: SW1(config-if)# **spanning-tree bpduguard enable**
 - Default: SW1(config)# **spanning-tree portfast [edge] bpduguard default**
 - Enables BPDU Guard on all PortFast-enabled ports.
 - Use **spanning-tree bpduguard disable** to disable it on specific ports.
- An err-disabled port can be re-enabled in two ways:
 1. Manual: **shutdown** and **no shutdown**
 2. Automatic: **ErrDisable Recovery**
 - SW1(config)# **errdisable recovery cause bpduguard**
 - In either case, make sure you fix the underlying problem that caused the port to be err-disabled.
- **BPDU Filter** prevents a port from sending BPDUs.
 - Unlike BPDU Guard, it does not disable the port if it receives a BPDU.
 - Per-port: SW1(config-if)# **spanning-tree bpdufilter enable**
 - The port will ignore any BPDUs it receives. **Use with caution!**
 - Default: SW1(config)# **spanning-tree portfast [edge] bpdufilter default**
 - Enables BPDU Filter on all PortFast-enabled ports.
 - If the port receives a BPDU, PortFast and BPDU Filter are disabled, and it operates as a normal STP port.
 - Use **spanning-tree bpdufilter disable** to disable it on specific ports.

Root Guard

- When selecting a LAN's root bridge, you should consider the following:
 - Optimal traffic flow
 - minimize latency
 - minimize congestion
 - Stability and reliability
- Within your own LAN, you can easily control the root bridge by setting its priority to 0.
 - There are cases where you might connect your switches to other switches outside of your control (e.g. service provider + client).
- **Root Guard** can be configured on specific ports to prevent them from accepting **superior BPDUs** from switches outside of your control.
- Use SW(config-if)# **spanning-tree guard root** to enable Root Guard on a port.
 - There is no command to enable it by default from global config mode.
- Root Guard prevents a port from becoming a root port if it receives a superior BPDU.
 - If the port receives a superior BPDU, it becomes Broken (BKN) / Root Inconsistent (ROOT_Inc).
- If the port stops receiving superior BPDUs, it will automatically recover.

Loop Guard

- **Loop Guard** protects the network from loops by blocking a port if it unexpectedly stops receiving BPDUs.
 - A software bug preventing a switch from sending BPDUs
 - A hardware issue causing a unidirectional link.
- A **unidirectional link** is a network link where data transmission occurs in only one direction.
 - Typically caused by Layer 1 issues on fiber-optic cables.
 - If the connected devices don't detect the issue and disable their interfaces, it can result in a unidirectional link.
 - If a Root or Non-Designated port stops receiving BPDUs, it will become a Designated port, potentially causing a Layer 2 loop.
- If a **Loop Guard**-enabled port stops receiving BPDUs, it enters the **Broken** (*Loop Inconsistent*) state, effectively disabling the port.
 - If it starts receiving BPDUs again, it will be automatically re-enabled.
- Loop Guard can be enabled in two ways:
 - Per-port: SW3(config-if)# **spanning-tree guard loop**
 - Default: SW3(config)# **spanning-tree loopguard default**
 - This enables Loop Guard on all ports
 - Use SW3(config-if)# **spanning-tree guard none** to disable it on specific ports if needed.
- **Loop Guard** and **Root Guard** are mutually exclusive.
 - If Loop Guard is configured on a port and you then configure Root Guard, Loop Guard will be disabled on the port (and vice-versa).
 - If Loop Guard is enabled by default (**spanning-tree loopguard default**) and you then configure Root Guard on a port, Loop Guard will be disabled on the port.